

Worst case execution time analysis (WCET)

Problem statement and motivation

Real time systems must perform their tasks (hardware or software) within fixed deadlines. If they don't, it is system failure. We use WCET analysis methods to find the upper bounds of the execution time that the program will take. This gives us an overview of how long the program will take in the worst case. There are many WCET analysis methods. But they all have their pitfalls and challenges. To ensure that a system completes its task within deadline, engineers need a reliable estimate of the WCET.

Core technical idea

The core idea is to study the different methods used to calculate the WCET. Also compare the different methods and look at their trustworthiness and pitfalls. The main goal is to become aware of the limitations of the methods to make a good selection of a method that best fits the user needs. There are 3 types of analysis methods; static, measurement-based and hybrid. They are further divided into 2 categories; deterministic and probabilistic. Static methods use a hardware timing model and a Control Flow Graph to study possible execution paths, loops, and branches without running the program. Measurement based methods execute the program on real hardware and use the highest observed execution time as the basis for the estimate. Probabilistic methods use repeated measurements and statistical analysis to produce a WCET distribution.

Analysis Methods

- **Static techniques:** Estimate WCET by analyzing the program structure and hardware timing model without running the program.
- **Measurement based techniques:** Estimate WCET using execution time measurements collected by running the program on real hardware.
- **Hybrid techniques:** Combine measured execution times with static information about the program's possible execution paths.
- Each technique can be deterministic, producing one WCET value, or probabilistic, producing a WCET value with a probability of exceeding it.
- This creates six methods: SDTA, SPTA, MBDTA, MBPTA, HYDTA, and HYPSTA.

Method	How it works	Main advantage	Main limitation
Static	Analyses the program and hardware model without running the program	Can provide safe upper bounds	Depends on correct models and becomes difficult for complex hardware
Measurement based	Runs the program on real hardware and records execution times	Uses real execution data and is practical	The real worst case may not be tested
Hybrid	Combines measurements with static program information	Improves coverage compared with measurements alone	Still depends on test quality and correct program information

Assumptions and scope limitations

- **Static methods:** These methods assume that the hardware timing model, processor documentation, and program flow information are correct. Their use becomes difficult for complex software and modern multicore processors because caches, pipelines, and shared resources are hard to model accurately.
- **Measurement based methods:** These methods assume that the test platform matches the real system and that the selected test inputs produce the longest relevant execution times. Their main limitation is that not every possible input and hardware condition can be tested, so the real worst case may remain unobserved.
- **Hybrid methods:** These methods assume that measured execution times can be safely combined with static information about program paths. They improve path coverage, but may still miss complex paths and cannot fully reproduce cache or memory interactions between program parts measured in separate runs.

Runtime, memory, scalability and resource aspects

- **Static methods:** The analysis effort increases with program complexity and hardware features such as caches, pipelines, and shared buses. They may require large timing models and many flow constraints, making them difficult to scale to complex multicore processors. Static methods can be expensive because detailed processor models and manual program annotations are needed.
- **Measurement based methods:** These methods can require many test inputs and repeated program runs. More testing increases runtime, storage needs, and engineering effort, but still cannot guarantee that every relevant condition has been covered. Measurement based methods are usually easier to apply, but large test campaigns can still require considerable time and labour.

- **Hybrid methods:** Hybrid techniques reduce some testing effort by using static path information, but they still require measurements, program models, and additional analysis. Their scalability is limited by complex software paths and multicore interference. Hybrid methods try to reduce costs by combining both approaches.

In all three methods, shared resources such as caches, buses, and memory controllers can increase execution time and make WCET estimation more difficult.

Questions

Which method is most suitable for modern multicore processors?

How can we measure the trustworthiness of an estimated WCET?